Ensemble Methods: Boosting vs. Bagging

An Empirical Study

Shahin Alakparov Narmina Ibragimova Hasan Mammadov

AI Academy, National AI Center - Summer 2026

Motivation & Central Question

- Ensemble learning combines weak learners to reduce generalisation error beyond any single model.
- Two dominant paradigms exist with *fundamentally different* error-reduction strategies:
  - **Boosting** - sequential, additive model that *targets bias*: each round re-weights misclassified examples.
  - **Bagging** - parallel, independent trees that *target variance*: averaging decorrelated hypotheses.

  Central question: *Under what conditions does boosting outperform bagging, and vice versa - and why?*
- All algorithms implemented **from scratch in NumPy**: CART, AdaBoost (SAMME), and Random Forest.
- Scikit-learn serves only as a correctness baseline, *not* as a model source.

Datasets

Dataset	N	P	Classes	Key Challenge
Breast Cancer Wisconsin	569	30	2	Small, low-noise, well-separated classes
Adult Income (Census)	45 222	100	2	Large, class imbalance ($\approx$76% negative)
MNIST 3-vs-8	6 000	784	2	High-dimensional, visual digit representation
Forest Covertype	30 000	54	7	Multiclass, severe imbalance (0.457% minority, 137/30,000)

- The four datasets span orthogonal difficulty axes: *size*, *dimensionality*, *class balance*, and *multiclass imbalanced complexity*.
- All experiments use fixed seeds for reproducibility.

Methods: CART, AdaBoost, Random Forest

1. Decision Tree (CART)

- Binary splits maximising information gain:

$$\Delta I = I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R)$$

- Sort + `np.cumsum` sweep: $O(N \log N \cdot P)$ instead of naive $O(N^2 P)$.
- Weighted Gini/entropy — needed for AdaBoost's per-sample weights.
- Stops on: max depth, min leaf size, zero gain, identical features.

2. AdaBoost (SAMME)

Each round m: fit stump h_m on weights $w^{(m)}$, then

$$\epsilon_m = \sum_i w_i^{(m)} \mathbf{1}[h_m(x_i) \neq y_i]$$

$$\alpha_m = \ln \tfrac{1-\epsilon_m}{\epsilon_m} + \ln(K-1)$$

$$w_i^{(m+1)} \propto w_i^{(m)} e^{\alpha_m \mathbf{1}[h_m(x_i) \neq y_i]}$$

Early-stop when $\epsilon_m \geq (K-1)/K$.

3. Random Forest

- Bootstrap sample of size N per tree; OOB samples give a free validation set.
- $\lfloor \sqrt{P} \rfloor$ random features per node split — decorrelates trees, which is what reduces variance.
- Trees trained in parallel via `multiprocessing.Pool`.
- Prediction: majority vote; `predict_proba` averages tree probabilities.

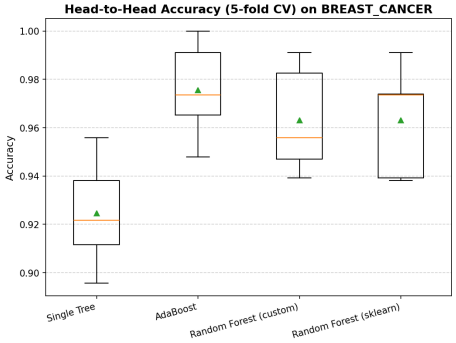
All three implemented from scratch in NumPy — see Exp. 1–3 for the corresponding empirical results and figures.

Exp. 1 - Correctness: Custom Tree vs. Sklearn

Why this matters:
Agreement within 2% limits provides empirical evidence that our from-scratch implementation is algorithmically equivalent to scikit-learn.

Dataset	Custom	Sklearn	$\lvert\Delta\rvert$
Breast Cancer	0.9469	0.9469	0.0000
Adult Income	0.8092	0.8105	0.0013
MNIST 3-vs-8	0.9292	0.9317	0.0025
Covertype	0.7802	0.7780	0.0022

Interpretation: All test accuracy differences $\leq 0.0025 \ll 0.02$ specification. Gaps arise from split search tie-breaking.



CV folds (Exp 4) show our custom tree generalises comparably to sklearn baseline.

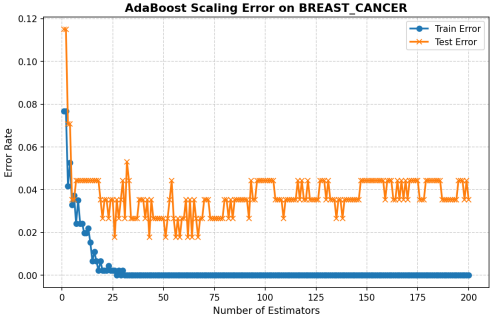
Exp. 2 - AdaBoost Scaling: #Stumps vs. Accuracy

What we measure: Staged train and test accuracy as T (number of stumps) grows from 1 to 200.

Expected behaviour: AdaBoost's additive model converges because each stump contributes a small correction. The exponential weight update ensures that already-correct examples receive less attention.

Key findings:

- Accuracy saturates around $T \approx$ 50-80 on Breast Cancer.
- Small train-test gap — no catastrophic overfitting due to low capacity (depth-1).
- Early stopping triggers on Adult Income where severe imbalance pushes error rate ϵ_m near $(K-1)/K$.

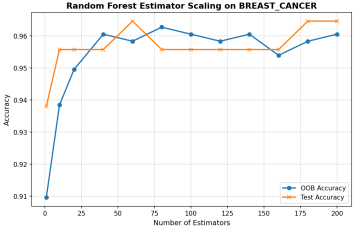


Breast Cancer. Near-zero train-test gap confirms depth-1 stumps avoid memorisation, yet aggregate strong generalisation.

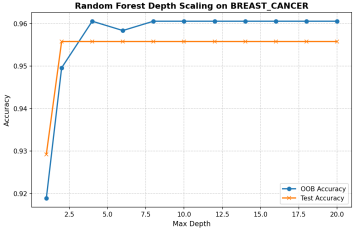
Exp. 3 - Random Forest: Trees & Depth Scaling

(a) Varying #trees (T), depth $= \infty$: OOB accuracy converges quickly ($T \approx 60$). Bounded by mean inter-tree correlation $\bar{\rho}$: adding more trees offers negligible decorrelation benefit.

(b) Varying max-depth, $T = 100$: Accuracy plateaus at depth 4–6 on Breast Cancer, but rises up to depth 20 on Covertype, proving that optimal tree depth is dataset-complexity dependent. Individual tree overfitting is cancelled out by averaging (bagging converts variance to diversity).



(a) OOB accuracy converges before $T = 100$.

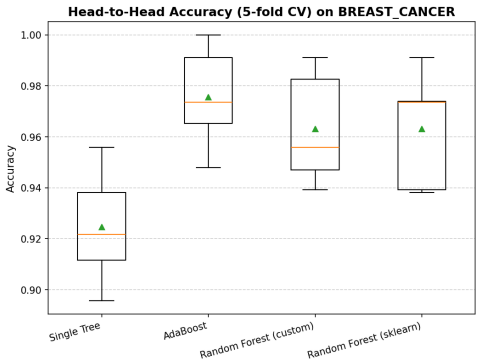


Exp. 4 - Head-to-Head: Binary Datasets

Protocol: Stratified 5-fold CV, $T = 100$ estimators, measuring Accuracy, Macro-F1, and AUC-ROC.

Key findings (Binary):

- **Breast Cancer:** AdaBoost wins on accuracy (0.9755 ± 0.0186) — sequential re-weighting excels on clean, low-noise data.
- **Adult Income:** Custom RF outperforms AdaBoost on AUC-ROC (0.9169 vs. 0.9073) - bootstrap averaging handles the class imbalance more robustly than weight amplification.
- **MNIST 3-vs-8:** RF dominates (0.9808 ± 0.0035) - random subspacing effectively handles 784 noisy pixel features; AdaBoost struggles with high dimensionality



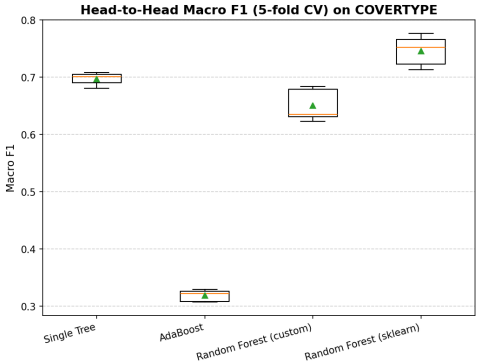
5-fold CV confirms both ensembles substantially outperform the single tree on Breast Cancer.

Exp. 4 - Head-to-Head: Covertype Multiclass

Severe Imbalance Challenge: Covertype subset ($N = 30,000$, 7 classes) has a minority class of just 0.457% (137 samples).

Macro-F1 Collapse findings:

- **AdaBoost collapse:** Macro-F1 drops to 0.3186 ± 0.0093 because its stumps ignore rare classes to maximize global accuracy.
- **Random Forest resilience:** RF achieves 0.7457 ± 0.0245 (sklearn) and 0.6501 ± 0.0258 (custom). Bootstrapping preserves minority representation.
- **RF Ensemble Gap:** The 4.08% custom-to-sklearn RF gap is an ensemble-level artifact (single tree matched within 0.22%).



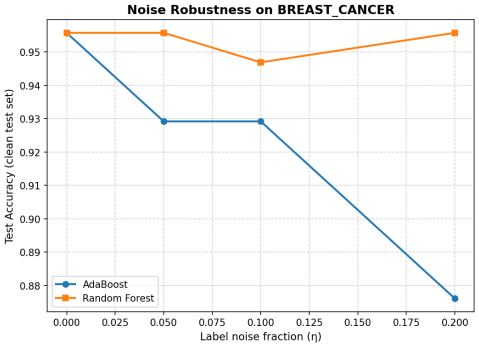
Covertype Macro-F1 comparison. AdaBoost collapses under severe imbalance, whereas Random Forest maintains high Macro-F1.

Exp. 5 - Noise Robustness: Binary Datasets

Protocol: Randomly flip $\eta \in \{0, 0.05, 0.10, 0.20\}$ fraction of training labels. Measure resulting test accuracy.

Mechanistic explanation:

- **AdaBoost vulnerability:** Weight update exponentially amplifies focus on misclassified examples. Truly mislabelled samples receive massive weights, forcing stumps to fit noise and warping decision boundaries.
- **Random Forest resilience:** Bootstrap sampling dilutes corrupted labels — a flipped label appears in $\approx 63\%$ of trees. Averaging acts as a low-pass filter to reject noise.



Breast Cancer: AdaBoost drops -7.96% at $\eta = 0.20$ while RF loses 0.00%. The bias-reduction power of AdaBoost becomes a liability under label corruption.

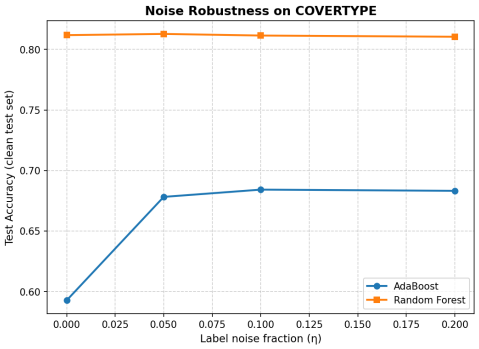
Exp. 5 - Noise Robustness: Covertype Anomaly

Regularisation Anomaly: On Covertype, introducing label noise actually *improves* AdaBoost test accuracy:

$$0.5927 \rightarrow 0.6782 \text{ at } \eta = 0.05 \ (+8.5\% \text{ gain})$$

Recall-based Diagnostics:

- **At $\eta = 0.00$:** AdaBoost chases rare minority classes (achieving 40.0% recall on Class 5, 7.73% on Class 6) at the cost of distorting dominant boundaries (Class 1 recall is only 53.6%).
- **At $\eta > 0$:** Injected noise makes rare classes unlearnable. AdaBoost "gives up" on them (Class 5 recall drops to 1.1%, Class 6 to 0%) and focuses entirely on dominant classes (Classes 0, 1, 2).
- Dominant recalls rise (Class 1 $\rightarrow$ 74.8%; Class 2 $\rightarrow$ 71.0%), yielding overall accuracy gains.



Covertype: Label noise acts as a regulariser under severe imbalance by forcing the booster to abandon unlearnable rare classes.

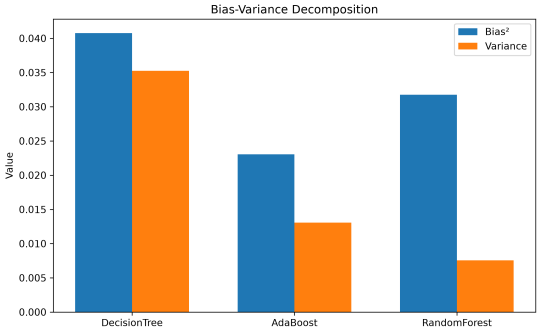
Exp. 6 - Bias-Variance Decomposition

Protocol: $B = 100$ bootstrap replicates of the Breast Cancer training set. Decompose expected error as:

$$\mathbb{E}[\text{err}] \approx \text{Bias}^2 + \text{Variance} + \text{Noise}$$

Model	Bias^2	Variance
Single Tree	0.0408	0.0352
AdaBoost	**0.0231**	0.0131
Random Forest	0.0317	**0.0076**

Interpretation: AdaBoost's sequential correction mechanism reduces Bias^2 by 43% relative to a single tree, at the cost of higher variance. RF halves variance (-78%) while reducing bias by a moderate amount - perfectly consistent with the bagging theory.



Stacked Bias^2/Variance bars confirm the fundamental trade-off: AdaBoost targets bias; Random Forest targets variance.

Exp. 7 - Unsupervised Structure

- **PCA Scree:** 7 of 30 principal components explain $\geq 90\%$ of variance - the data lies on a low-dimensional manifold, implying that many raw features are correlated/redundant.
- **K-Means** ($k = 2$, ARI$= 0.6536$): substantial but imperfect agreement with true labels - PCA space is approximately separable, validating the linear structure of the feature space.
- **DBSCAN** (ARI$= 0.0653$, 13.2% noise): density-based clustering fails here because Breast Cancer data lacks the sharp density boundaries DBSCAN requires - the two classes form overlapping, ellipsoidal rather than compact clusters.

PCA scatter (PC1 vs PC2) coloured by true label. Clear but imperfect linear separation explains K-Means success and DBSCAN failure.

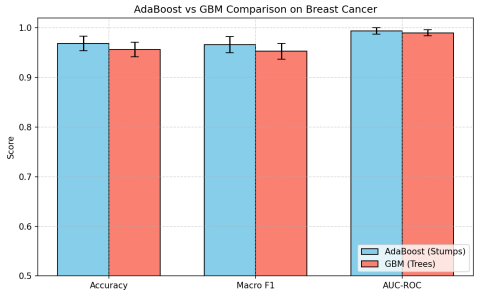
Bonus: GBM vs. AdaBoost & SAMME vs. SAMME.R

GBM vs. AdaBoost (5-fold CV):

- AdaBoost outperforms custom GBM across all binary datasets.
- Custom GBM uses higher-capacity depth-3 trees but is under-converged at learning rate 0.1 and 100 estimators.
- Stumps are extremely low capacity, allowing AdaBoost to quickly extract signal and converge within the fixed round budget.

SAMME vs. SAMME.R (Iris, 80 rounds):

- Both algorithms achieve 90% test accuracy.
- SAMME.R log-loss is **0.1322** vs. SAMME's 0.6648.
- SAMME.R produces far better calibrated class probabilities because it updates on soft probability estimates rather than hard votes.



AdaBoost vs. GBM across CV folds (Breast Cancer). AdaBoost consistently converges faster.

Discussion: When Does Each Win - and Why?

Boosting wins when:

- Data is clean and low-noise (label corruption is catastrophic).
- The dominant error source is high bias - a single model systematically underfits.
- Dataset is small/structured: sequential weight updates are data-efficient.
- Evidence: Breast Cancer (Bias^2 reduction -43%, lowest total error 0.0362 vs. 0.0393).

Bagging wins when:

- Labels are noisy: bootstrap averaging dilutes corrupted examples.
- Feature space is high-dimensional: random subspacing decorrelates trees effectively.
- The dominant error source is high variance - individual trees overfit.
- Evidence: MNIST (RF 0.9808 vs. AdaBoost 0.9505); noise robustness (0.00% drop vs. -7.96%).

Neither algorithm uniformly dominates. The correct choice depends on data cleanliness, dimensionality, and whether the dominant error source is bias or variance.

Conclusion

- Implemented CART, AdaBoost (SAMME/SAMME.R), and Random Forest **from scratch in NumPy**; all match sklearn within $\leq 0.25\%$ accuracy, confirming algorithmic correctness.
- **AdaBoost** reduces Bias^2 more aggressively (-43% vs. single tree) through sequential re-weighting - at the cost of instability under label noise.
- **Random Forest** reduces Variance more dramatically (-78% vs. single tree) through decorrelated bootstrap averaging - and is robust to 20% label corruption.
- Practitioners' rule of thumb: use *boosting* on clean, structured data where underfitting is the bottleneck; use *bagging* when data is noisy, high-dimensional, or class-imbalanced.

Thank you - Questions?